Neurosymbolic Program Synthesis: From Enumerative Search to LLM-Guided Methods

An Annotated Bibliography

CSE 590: Advanced Programming Languages

Vanshika Singla (vanshiii@umich.edu)
University of Michigan

April 20, 2026

Narrative

Program synthesis means getting a computer to write a program for you, given some description of what it should do. That description could be input-output examples, a natural language prompt, or a formal logical specification. The problem is harder than it looks. Even a small set of examples matches a huge number of programs, most of them wrong, and there is no obvious way to figure out which one the user actually wanted. For most of computing history this stayed a theoretical problem. Getting it to work in practice, and then ship in a product used by millions of people, took about two decades.

Solar-Lezama et al.'s SKETCH system [6] is where most people start when tracing this history. Before SKETCH, every synthesis tool needed hand-coded domain rules, which made each one narrow and hard to reuse. SKETCH changed this by letting the programmer write a partial program with holes where the hard parts go, and using a SAT-based search to fill those holes in. No domain knowledge is needed because the search is purely combinatorial, and any completed program is provably correct relative to the specification. SKETCH also introduced CEGIS, the counterexample-guided inductive synthesis loop, where a failed candidate generates a counterexample that rules out similar wrong answers in the next round. CEGIS shows up beyond synthesis too, in program verification, test generation, and compiler optimization, and nearly every synthesis system after SKETCH uses some version of it.

Gulwani's FlashFill [4] is where synthesis stopped being a research problem and started running on people's laptops. Excel users spend a lot of time reformatting text columns manually because writing formulas is too hard. FlashFill lets them type a couple of examples and synthesizes the transformation automatically. The key contribution is version space algebras, a data structure that represents all programs consistent with the given examples at once instead of trying them one by one, which is what makes the whole thing fast enough to run while you type. On 175 benchmarks from real Excel users, it got the right answer from a single example 74% of the time and ran in under one second on everything. It shipped in Excel 2013 and won the most influential paper award at POPL in 2021. The thing worth noting is that FlashFill was evaluated on real user tasks, not artificial benchmarks, and success meant the system actually shipped and got used.

By 2017 the field had grown messy enough that Gulwani, Polozov, and Singh [5] wrote a survey to organize it. People in programming languages, machine learning, and HCI were all working on related problems but using different terminology and mostly unaware of each other. The survey organizes everything around three questions: how does the user express intent, what is the program space being searched, and how does

search work. It covers Manna and Waldinger's deductive synthesis from the 1970s all the way through early neural methods. The three-axes framework it introduced became the standard vocabulary for describing new synthesis work.

DreamCoder [2], published at PLDI in 2021, is the most interesting pre-LLM system in this set. The problem it addresses is that every prior system started from scratch on each new task. DreamCoder instead builds a library of reusable functions alongside a neural search policy, both getting better over time through a wake-sleep algorithm. After solving a batch of tasks, it compresses the solutions to find common patterns and adds them to the library as named primitives, then retrains the neural policy on the updated library. Starting from only basic recursion, it eventually rediscovered map, filter, and fold on its own. What makes this different from just better benchmark numbers is that the system actually accumulated structure over time rather than solving each task as a one-off problem.

Codex [1], published by Chen et al. in 2021, changed the conversation. A large language model fine-tuned on GitHub code could generate programs from natural language with no explicit search, no DSL, and no formal specification at all. On HumanEval, 164 hand-written Python problems with unit tests, Codex got 28.8% correct with a single sample and 70.2% when allowed 100 samples. This became the foundation for GitHub Copilot. The catch is that Codex has no notion of correctness. It predicts likely code, not verified code, and there is no mechanism for it to know when it is wrong.

LILO [3], published at ICLR 2024 by Grand et al., tries to get the best of both. It takes DreamCoder's wake-sleep framework and replaces the expensive enumerative search with LLM-guided synthesis, while keeping the compression and library learning unchanged. The one new addition is automatic documentation: after each compression step, the LLM generates readable names and descriptions for newly discovered functions. Without this, giving the LLM the compressed abstractions actually hurt performance, dropping solve rates by over 30 points on the REGEX domain. LILO outperformed DreamCoder by 33.14 points on REGEX and 20.42 points on LOGO, and beat LLM-only baselines by up to 16.82 points on LOGO.

None of these systems solve the whole problem. Classical methods need hand-designed DSLs for each new domain. Neural methods fail without warning. Neurosymbolic systems like LILO still need hand-designed primitives to start. Work from 2024 shows hybrid approaches solving around 80% of standard benchmarks, but a system that works across arbitrary new domains without significant upfront engineering does not exist yet.

Updated Implementation Plan

After reading through the papers, my implementation idea has gotten more specific. Originally I was thinking of a basic enumerative synthesizer with some LLM component bolted on, but reading LILO and DreamCoder changed how I think about the problem. The interesting question is not just whether LLMs help but when they help and why. LILO's ablation results showing that giving an LLM unnamed abstractions actually hurts performance was surprising and something worth exploring at a smaller scale.

The plan is to build two things in Python. First, a bottom-up enumerative synthesizer that searches over a small arithmetic and string expression language given input-output examples. The expression language will cover arithmetic operations like addition and multiplication, string operations like concatenation and slicing, and simple conditionals, which is expressive enough to cover interesting tasks but constrained enough to make enumeration feasible. This is the classical baseline, directly from the ideas in Solar-Lezama et al. and Gulwani. Second, an LLM-guided version that uses an LLM to rank or generate candidates instead of enumerating blindly, loosely following LILO's dual-system approach but at a much smaller scale. The OpenAI API will be used for the LLM component.

Both will be evaluated on the same set of around 20 benchmark tasks covering simple arithmetic transformations and string manipulations. The metrics are number of candidates explored before finding a correct program and time to solution. The goal is to see whether LLM guidance actually reduces search cost, and whether there are task types where brute force enumeration wins. DreamCoder's library learning component was considered but ruled out as too complex for the scope. Getting the baseline and LLM comparison right is the right 20–24 hour target.

References

[1] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, *et al.*, "Evaluating large language models trained on code," *CoRR*, vol. abs/2107.03374, 2021. [Online]. Available: https://arxiv.org/abs/2107.03374

> The question this paper is asking is whether you even need classical synthesis machinery if you have a large enough language model trained on enough code. Prior systems needed formal specs, carefully designed DSLs, or domain-specific engineering. Codex just trains on GitHub and asks the model to write code from a description. Codex is GPT-3 fine-tuned on publicly available GitHub code. Given a natural language prompt or a function docstring, it predicts likely token continuations. No explicit search, no specification language, no domain knowledge. You can sample many candidates and filter them with test cases. From a synthesis standpoint, Codex has no notion of a specification and gives no correctness guarantee. It sits at the far end of the intent axis from the survey, where the user just describes what they want in natural language and the system figures out the rest. The paper introduces HumanEval, 164 hand-written Python problems each with unit tests. Codex-12B got 28.8% correct with one sample and 70.2% when allowed 100 samples with test filtering. GPT-3 with no code fine-tuning got essentially 0%, so the code training was clearly doing the work. These numbers were well above anything prior and set a new practical benchmark the field is still measuring against. The core problem is that the model has no way to know when it is wrong. It produces code that looks right and passes your examples but can fail on edge cases you did not think to test. It also struggles on problems requiring novel algorithms or precise reasoning. The paper flags security vulnerabilities in generated code as a real concern. None of this gets fixed with a bigger model or more data. It is a mismatch between what a language model does and what synthesis actually needs.

[2] K. Ellis, C. Wong, M. I. Nye, M. Sablé-Meyer, L. Morales, L. B. Hewitt, L. Cary, A. Solar-Lezama, and J. B. Tenenbaum, "Dreamcoder: bootstrapping inductive program synthesis with wake-sleep library learning," in *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI)*. ACM, 2021, pp. 835–850.

> Every prior synthesis system started from scratch on each new task. That works for simple problems but does not scale, and it ignores how programmers actually get better over time. Neural systems can generalize across tasks but give no correctness guarantees and tend to struggle with compositional program structure. DreamCoder asks whether a synthesis system can actually improve with experience, building up reusable concepts the way a programmer does. The answer is library learning plus a neural search policy, both trained together through a wake-sleep algorithm borrowed from Bayesian inference. During wake, the system solves a batch of tasks using neurally-guided enumerative search, with a recognition network scoring which programs to try first. During sleep, it compresses the programs it found to pull out shared substructures, adds them to a library as named functions, and retrains the neural policy on the updated library. Each round starts with better building blocks and a smarter search

guide. Across eight domains, DreamCoder rediscovered map, filter, and fold starting from only basic recursion and list primitives. It discovered expressions for Newton's and Coulomb's laws from raw physics data starting from basic arithmetic. It beat pure enumeration, pure neural baselines, and prior library learning systems in every domain. On text editing tasks, the full system produced clean interpretable regex patterns while ablations without the library produced much messier results, directly showing library learning was driving the improvement. Enumerative search across eight domains over many iterations gets expensive fast. The system also needs enough training tasks to get library learning off the ground, and if training and test tasks look different the learned library does not transfer well. LILO later addressed the search cost by swapping in LLM-guided synthesis, though that introduced dependence on an external language model.

[3] G. Grand, L. Wong, M. Bowers, T. X. Olausson, M. Liu, J. B. Tenenbaum, and J. Andreas, "LILO: learning interpretable libraries by compressing and documenting code," in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: https://openreview.net/forum?id=TbpJIVEFNs

DreamCoder showed library learning works but enumerative search makes it slow. Codex showed LLMs can generate code fast but without any way to accumulate reusable knowledge or check correctness. LILO tries to get both at once. It takes DreamCoder's wake-sleep framework and swaps out enumerative search in the wake phase for LLM-guided synthesis. The LLM generates candidate programs from natural language task descriptions, which is much faster than exhaustive enumeration. The sleep phase stays the same as DreamCoder, same compression, same abstraction. The new piece is automatic documentation: after each compression step, the LLM generates readable names and docstrings for the newly discovered library functions. This turned out to matter a lot. Without readable documentation, giving the LLM the compressed abstractions actually hurt performance, dropping solve rates by over 30 points on REGEX in ablation experiments. The LLM just ignores functions it cannot understand semantically. With auto-documentation it reuses them. On three domains from DreamCoder, LILO beat DreamCoder by 33.14 points on REGEX, 20.42 points on LOGO, and 2.26 points on CLEVR where DreamCoder was already at 94.5%. It also beat LLM-only baselines by up to 16.82 points on LOGO. LILO still needs a hand-designed set of domain primitives to initialize the library. That means it cannot start from scratch in a genuinely new domain without significant upfront work. It also depends on a capable LLM which costs money and adds latency. The strong results are on three domains and it is not clear how well this generalizes beyond them.

[4] S. Gulwani, "Automating string processing in spreadsheets using input-output examples," in *Proceedings of the 38th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. ACM, 2011, pp. 317–330.

Most people who use spreadsheets spend a lot of time manually fixing up text columns. Splitting names, extracting dates, combining fields. These are things non-programmers do all the time and struggle with. The synthesis problem here is specific: the user can only give one or two examples, the system has to figure out the right rule fast enough to run while they type, and it has to deal with the messy ways real users give examples. Gulwani built a DSL for string manipulation that is expressive enough to cover real tasks but small enough to keep search fast. The main algorithmic idea is version space algebras, a data structure that represents all programs consistent with the given examples simultaneously instead of trying them one by one. This is what makes the system fast enough to run interactively. When examples are ambiguous

and multiple programs would give different answers on a new input, the system asks for one more example rather than guessing. On 175 benchmarks from real Excel users and product teams, FlashFill got the right answer from a single example in 74% of cases and converged in a few examples on the rest. It ran under one second on every benchmark. It shipped in Microsoft Excel 2013 and reached hundreds of millions of users. It won the most influential paper award at POPL in 2021. FlashFill only works on strings. Numeric operations, table transformations, anything outside strings needs a completely separate system. The version space approach also does not transfer to other domains without basically rebuilding everything. There is also a subtler problem: if the user's examples are ambiguous, FlashFill can produce a program that passes all the examples but does the wrong thing on new inputs, with no way for the user to know. Microsoft's PROSE framework later extended these ideas to more data types.

[5] S. Gulwani, O. Polozov, and R. Singh, "Program synthesis," *Foundations and Trends in Programming Languages*, vol. 4, no. 1–2, pp. 1–119, 2017.

By 2017 synthesis research had spread across programming languages, machine learning, and HCI, but people in each area were mostly unaware of what the others were doing and using different terminology for the same ideas. The survey was written to give the field a shared vocabulary. It organizes everything around three questions. How does the user express what they want? Options range from formal logical specs to input-output examples to natural language to partial programs. What programs are being searched? Could be a grammar, a type system, or a learned model. How does search work? Enumerative search, constraint solving, deductive inference, version spaces, stochastic methods, neural prediction. Mapping existing work onto these three axes shows which combinations have been tried and where the gaps are. The coverage goes from Manna and Waldinger's deductive synthesis in the 1970s through inductive logic programming, constraint-based methods, programming by example, and early neural approaches. It pulls together work from PL, ML, and HCI that had mostly developed separately. The three-axes framework became standard vocabulary for the field and the survey is still assigned in graduate courses because nothing comparably comprehensive exists for this time period. The obvious problem is timing. Neural methods were just getting serious in 2017 so the coverage there is thin. There is no treatment of large language models, no library learning, no neurosymbolic systems. The authors flag combining neural and symbolic approaches as a major open direction, and DreamCoder and LILO are both direct responses to that.

[6] A. Solar-Lezama, L. Tancau, R. Bodík, S. A. Seshia, and V. A. Saraswat, "Combinatorial sketching for finite programs," in *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2006, pp. 404–415.

Every synthesis tool before SKETCH needed hand-coded rules specific to its domain. If you wanted a synthesis tool for cryptography you built crypto rules from scratch. If you wanted one for networking you started over. There was no general approach, and searching the full space of programs without some constraint is just computationally hopeless. SKETCH changed this by having the programmer write most of the program and leave holes where the hard parts go. A SAT-based search fills in the holes. The programmer does not need to supply domain rules because the search is purely combinatorial, and any program the synthesizer produces is provably correct relative to the specification. SKETCH also introduced CEGIS, the counterexample-guided inductive synthesis loop. When a candidate program fails, the failure becomes a counterexample that rules out similar wrong answers in the next round.

CEGIS also shows up in program verification, test generation, and compiler optimization, so it spread well beyond synthesis. For evaluation, SKETCH synthesized a complete AES encryption implementation. The generated code ran in 21.3 ms for 50,000 encryptions. Hand-optimized OpenSSL ran in 19.6 ms. The original specification ran in 19,936 ms, over 1,000 times slower. So SKETCH got within 10% of expert-tuned code automatically. A MultiGrid stencil kernel finished in under 5 minutes. These were not toy programs. The main limitation is that the programmer still has to understand the algorithm well enough to write a meaningful sketch. That is a real barrier for non-experts. The system also only handles finite programs, so anything with unbounded loops is out of scope. And synthesis time goes up with the number and complexity of holes. Later work extended SKETCH to concurrent programs and CEGIS became a standard building block for a much wider range of synthesis systems.